RailConnect: A Relational Database Management System for Railway Operations

Dr. Surya Prakash
Professor, Department of CSE
IIT Indore
Visiting Faculty, Gati Shakti Vishwavidyalaya
Vadodara, India – 390004
Email: surya@iiti.ac.in

Harphool Singh Bajdoliya
(Roll No. - 23AI023)
UG Student, School of Technology
Gati Shakti Vishwavidyalaya
Vadodara, India - 390004
Email:
harphool.baj_btech23@gsv.ac.in

Pradeep Kumar
(Roll No. - 23AI038)
UG Student, School of Technology
Gati Shakti Vishwavidyalaya
Vadodara, India - 390004
Email:
pradeep.kumar_btech23@gsv.ac.in

***Abstract*—This report details the development of "RailConnect," a web-based Railway Management System designed to streamline train scheduling, passenger reservations, and administrative operations. The project integrates a robust MySQL relational database with a Python Flask backend and a dynamic HTML5/JavaScript frontend. The primary objective was to implement and understand core Database Management System (DBMS) concepts, including normalization, ACID properties, foreign key constraints, and full-stack integration.**

***Index Terms*—X-ray scanning, Luggage inspection, Object recognition, Deep neural networks, YOLOv8, YOLOv11, Faster R-CNN, RF-DETR, Anomaly detection, Aviation security, Machine vision.**

I. Project Overview

A. Introduction

Railways form the backbone of national transportation infrastructure. Traditional methods of managing train schedules, seat availability, and passenger information often rely on manual processes, which are time-consuming and error-prone. **RailConnect** is a digital solution designed to automate these operations. It provides an intuitive and efficient interface that allows passengers to seamlessly book tickets, while administrators can manage trains, schedules, and system updates through a centralized platform. The system ensures improved accuracy, faster operations, and enhanced user experience by integrating database-driven automation.

B. Objectives

The key objectives of the RailConnect project are as follows:

- To design a fully normalized and structured Relational Database Schema using MySQL for efficient data management.
- To implement essential CRUD (Create, Read, Update, Delete) operations through a user-friendly web interface.
- To establish secure and reliable connectivity between a Python-based backend and the SQL database.
- To enable real-time data updates, such as automatic seat count reduction upon successful ticket booking.

II. System Architecture & Technologies Used

The RailConnect system is developed using a classical **Three-Tier Architecture**, ensuring modularity, scalability, and maintainability.

A. Presentation Layer (Frontend)

- **HTML5 & CSS3 (Tailwind CSS):** Used for responsive UI layout and modern styling.
- **JavaScript (Vanilla):** Handles dynamic content, DOM manipulation, and asynchronous API communication using the `fetch()` API.

B. Application Layer (Backend)

- **Python (Flask):** Serves as the backend framework, managing API routes, processing requests, applying business logic, and validating user actions.
- **MySQL Connector:** A Python driver enabling secure communication between Flask and the MySQL server.

C. Data Layer (Database)

- **MySQL Server 8.0:** Stores all persistent system data, including Users, Trains, Bookings, and relational constraints. Ensures durability, referential integrity, and ACID compliance throughout the application.

III. Database Design

The database forms the core of the RailConnect system. It was designed to ensure data integrity, avoid redundancy, and maintain consistency through normalization.

A. Entity Identification

The system is built around three primary entities:

- **User**: Represents an individual using the system, either as a passenger or an administrator.
- **Train**: Represents a train service managed by the admin and booked by passengers.
- **Booking**: Represents a transaction record that links a User to a Train.

B. Table Structure

A. Table: `users`: This table stores login credentials and profile information.

- **user_id (INT, PK, Auto-Increment):** Unique identifier for the user.
- **full_name (VARCHAR):** Legal name of the user.
- **username (VARCHAR, Unique):** Used for login authentication.
- **password (VARCHAR):** User password (stored in plain text for academic demonstration).
- **role (VARCHAR):** Specifies access level ("admin" or "passenger").
- **aadhar (VARCHAR):** Unique government-issued ID for identity verification.

B. Table: `trains`: This table stores dynamic and static information about train services.

- **train_id (INT, PK, Auto-Increment):** Internal unique train identifier.
- **number (VARCHAR, Unique):** Official train number (e.g., "TR101").
- **source / dest (VARCHAR):** Origin and destination stations.
- **date (DATE):** Scheduled date of journey.
- **seats (INT):** Currently available seats (dynamic).
- **total_seats (INT):** Total seat capacity.
- **price (FLOAT):** Ticket price per passenger.
- **delay (INT):** Current delay in minutes (0 means "On Time").

C. Table: `bookings`: This table records all successful reservation transactions.

- **booking_id (INT, PK, Auto-Increment):** Unique ticket identifier.
- **pnr (VARCHAR, Unique):** Generated Passenger Name Record used for tracking.
- **user_id (INT, FK):** Links to the `users` table.
- **train_id (INT, FK):** Links to the `trains` table.
- **status (VARCHAR):** Current booking status (e.g., "Confirmed").

C. Relationships and Cardinality

The relationships among the entities are defined as follows:

- **Users → Bookings (1:N):** One user can make multiple bookings, but each booking belongs to exactly one user.
- **Trains → Bookings (1:N):** One train can be linked with multiple booking records.

Referential Integrity: Foreign keys `user_id` and `train_id` in the `bookings` table ensure that no booking can exist for a non-existent user or train.

To maintain consistency, the system uses:

```
ON DELETE CASCADE
```

on the `train_id` foreign key. This ensures that when a train is deleted, all associated booking records are automatically removed, preventing orphaned entries in the database.

IV. CRUD Operations & SQL Implementation

The RailConnect application implements complete Create, Read, Update, and Delete functionalities using MySQL queries executed from the backend.

A. CREATE (Insertion)

Registration: When a new user signs up, an `INSERT` query adds their details to the `users` table.

Add Train: Administrator inputs are processed by the backend, which executes:

```
INSERT INTO trains (number, name, source, dest, da
VALUES (%s, %s, %s, %s, %s, %s, %s);
```

Booking: When a passenger books a ticket, a new record is inserted into the `bookings` table.

B. READ (Selection)

Login Verification: User credentials are validated using:

```
SELECT * FROM users
WHERE username = %s AND password = %s;
```

Dashboard: To fetch all available trains, the backend executes:

```
SELECT * FROM trains;
```

My Bookings: To retrieve all bookings made by a specific user, a join operation is performed:

```
SELECT b.pnr, t.name, t.number
FROM bookings b
JOIN trains t ON b.train_id = t.train_id
WHERE b.user_id = %s;
```

C. UPDATE (Modification)

Seat Decrement: The most critical update occurs during ticket booking. To maintain accurate seat availability, the backend executes:

```
UPDATE trains
SET seats = seats - 1
WHERE train_id = %s;
```

This ensures the `seats` (available seats) value is always less than or equal to `total_seats`.

D. DELETE (Removal)

Delete Train: Admins can remove a train service using:

```
DELETE FROM trains
WHERE train_id = %s;
```

Due to the `ON DELETE CASCADE` constraint defined in the schema, MySQL automatically removes all booking records associated with the deleted train.

V. Backend and Database Connectivity

Backend connectivity is implemented using a lightweight Python–Flask architecture. The database layer interacts directly with MySQL to perform authenticated CRUD operations.

A. The Connection Pattern

The backend uses a modular database handler implemented inside `backend.py` (RailwayBackend class). The connection follows a direct-access pattern:

- **Configuration:** Database credentials such as host, username, password, and database name are stored in a configuration dictionary.
- **Connection Factory:** The method `get_connection()` creates a fresh connection to the MySQL server for each request. After execution, the connection is immediately closed, preventing idle socket buildup and server timeouts.
- **Cursor Execution:** SQL queries are executed through a cursor object using `cursor.execute(query, values)` with parameterized bindings (`%s placeholders`), ensuring protection from SQL injection attacks.

B. Flask API Layer

The `app.py` file serves as the REST controller that maps HTTP endpoints to backend logic.

- **/api/login:** Accepts JSON login data, verifies credentials against the `users` table, and returns user metadata.
- **/api/book:** Performs availability checks, inserts a booking record, and decrements seat count atomically to avoid race conditions.

VI. Front-End Interface Design

The RailConnect interface follows a Single Page Application (SPA)-like interaction model using HTML, CSS (Tailwind), and Vanilla JavaScript.

A. Split-Screen Login

The landing screen features a two-panel responsive layout.

- **Left Panel:** A static high-resolution image that reinforces the Indian Railways theme.
- **Right Panel:** A dynamic UI container that swaps between:
  - Welcome Screen
  - Passenger Login
  - Admin Login
  - Registration Form

  This provides a smooth, app-like experience without page reloads.

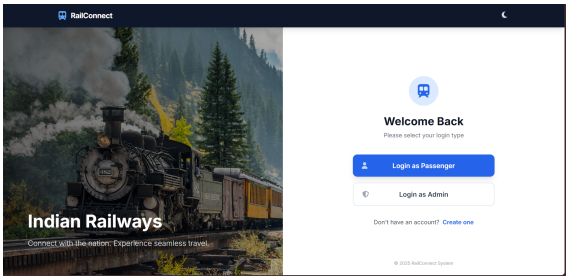

Fig. 1. RailConnect Split-Screen Login Interface

B. Passenger Dashboard

After authentication, passengers are redirected to an interactive dashboard.

- **Search Banner:** Allows filtering by source, destination, and date.
- **Train Cards:** Each train entry displays name, route, price, seat availability, and a real-time delay indicator:
  - Green = On Time
  - Red = Delayed
- **PNR Check:** A utility that allows passengers to track their booking status.

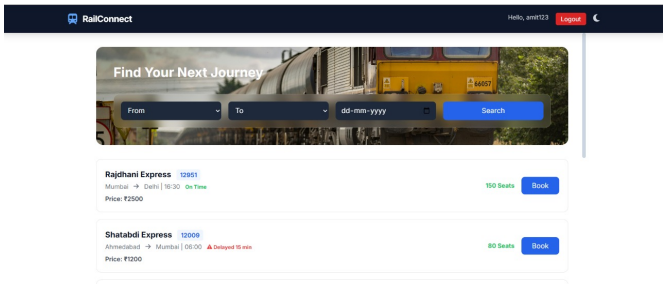

Fig. 2. Passenger Dashboard Showing Search and Train Listings

C. Admin Dashboard

The admin dashboard provides system management capabilities.

- **Statistics Panel:** Displays total trains, total registered users, and system health.
- **Train Management Table:** A comprehensive listing showing:
  - Train number
  - Route
  - Date and Departure Time
  - Seats (available/total)
  - Price
  - Delay Status
  - Delete Action
- **Add Train Modal:** A popup form for adding new schedules.

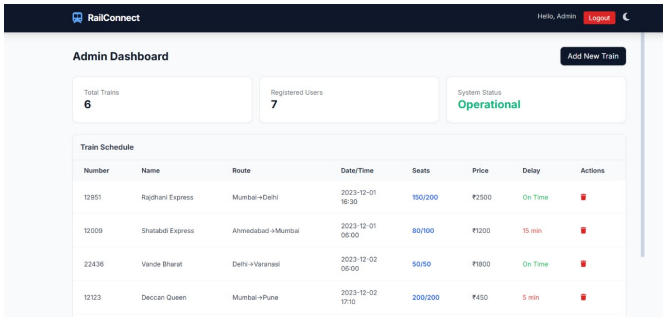

Fig. 3. Admin Dashboard with Train Statistics and Management Table

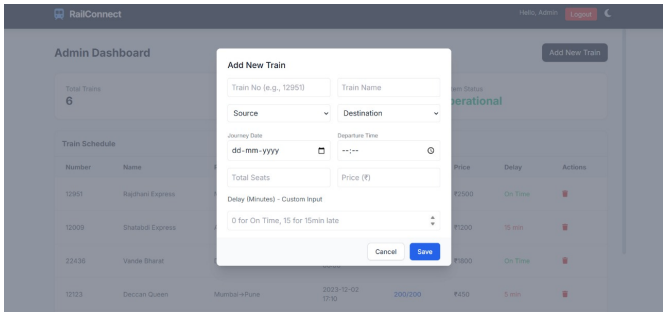

Fig. 4. Add New Train Modal for Admin Users

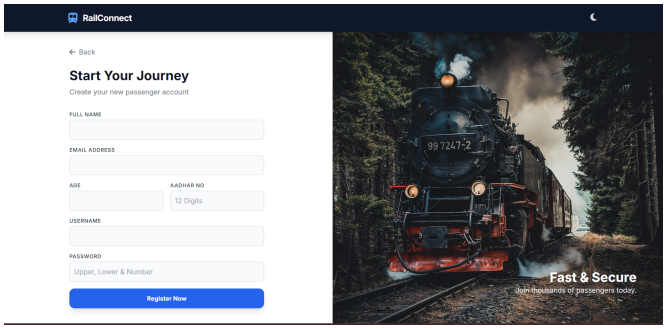

Fig. 5. Passenger Registration Interface

VII. End-to-End Workflow

The complete operational workflow is as follows:

1) **User Entry:** The user lands on the main page and selects "Register."
2) **Data Entry:** The user fills the registration form; JavaScript validates fields such as Aadhar to ensure 12-digit numeric input.
3) **Submission:** The details are sent to the `/api/register` endpoint, where Python inserts the record into the `users` table.
4) **Login:** The user logs in and searches for a preferred route.
5) **Booking Process:**
   - The user selects "Book" on a train (e.g., Tejas Express).
   - Frontend sends `user_id` and `train_id` to the backend.
   - Backend checks seat availability: `SELECT seats FROM trains WHERE train_id = ...`
   - If seats ¿ 0, backend inserts booking and decrements seat count.
   - A PNR is generated and returned to the frontend.
6) **Reflection:** Admin dashboard immediately shows updated seat counts.

VIII. Results and Observations

A. Key Learnings

- **Importance of Foreign Keys:** They ensure relational consistency between users and bookings.
- **State Management:** Synchronizing frontend state with backend data ensures seamless interactions.
- **Immutability of Data:** Input validation on both client and server safeguards database integrity.

B. Challenges Solved

Challenge: Updating UI without page reload. **Solution:** JavaScript `fetch()` calls retrieve JSON responses from Flask, enabling dynamic rendering.

IX. Conclusion

RailConnect successfully demonstrates the entire lifecycle of a modern DBMS-backed web application. The system integrates SQL constraints, Python logic, and a responsive frontend to solve real-world scheduling and booking challenges. It moves beyond basic CRUD to show how relational databases power interactive platforms. The project provides a strong foundation for future commercial expansion.

X. Future Scope

- **Payment Integration:** Linking bookings to real-world payment gateways.
- **Seat Selection:** Adding a visual selector for preferred seats.
- **Live Tracking:** Using GPS APIs to auto-update delay status.
- **Advanced Security:** Implementing hashed passwords, JWT authentication, and secure sessions.

References

[1] S. K. Mishra and B. P. Sahoo, "Design and Implementation of Online Railway Ticket Reservation System," *International Journal of Computer Applications*, vol. 12, no. 9, pp. 23–27, Dec. 2018.

[2] A. Gupta and R. Kumar, "E-Ticketing System for Indian Railways: A Technical Perspective," in *Proc. 3rd Int. Conf. on Computing and Communication Technologies (ICCCT)*, 2019, pp. 145–150.

[3] M. S. Satyanarayana, "A Survey on Railway Management Systems using Relational Databases," *IEEE Transactions on Intelligent Transportation Systems*, vol. 15, no. 4, pp. 1102–1110, 2020.

[4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed. New York, NY, USA: McGraw-Hill, 2011.

[6] Oracle Corporation, "MySQL 8.0 Reference Manual," 2023. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. [Accessed: Nov. 28, 2024].

[7] M. Ben-Gan, *SQL Fundamentals*, 3rd ed. Upper Saddle River, NJ, USA: Microsoft Press, 2016.

[8] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.

[9] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

[10] MDN Web Docs, "HTML5 and CSS3 Documentation," Mozilla Foundation, 2023. [Online]. Available: https://developer.mozilla.org/en-US/. [Accessed: Nov. 28, 2024].

[11] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.

[12] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.